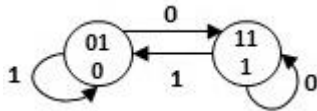
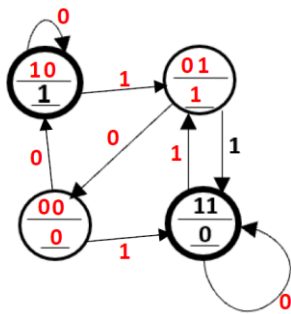


1.[18 points] Finite State Machine

(a)[12 points] Given the partially completed truth table and FSM diagram below, complete all the missing entries in the truth table and the FSM diagram. In each state circle, the top entry is S_1S_0 and the bottom entry is the value of Out. Make sure that you have labeled all missing states, inputs and outputs, and that you have added and labeled any missing transitions in the FSM.



Note that in the above diagram, '01' and '11' in the circle are encodes states, and '0' and '1' in the circle are outputs. In state '01', if input is '0', next state is '11' and output is '0'. In state '01', if input is '1', next state is '01' and output is '0'.

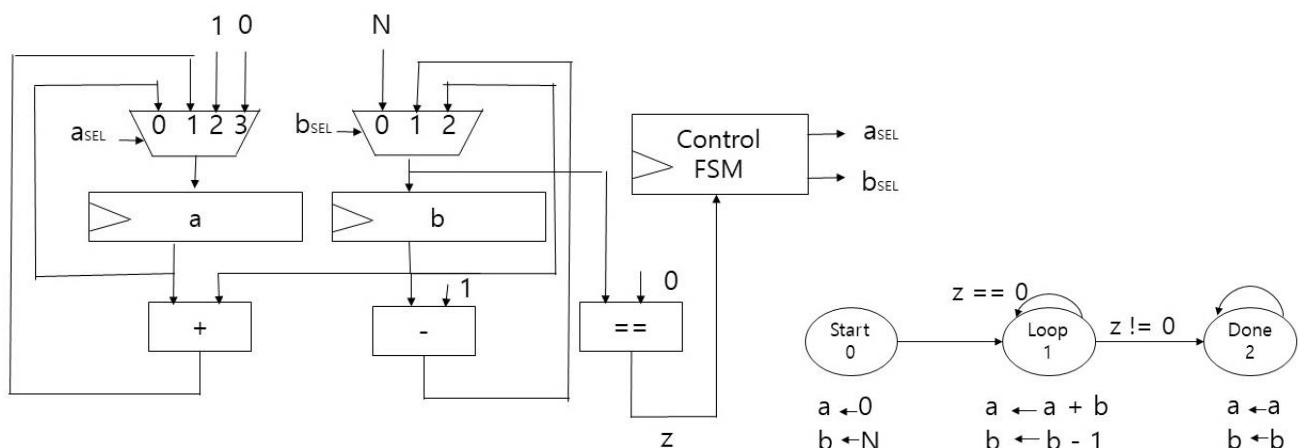


S_1	S_0	In	S_1'	S_0'	Out
0	0	0	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	1	1	1	1
1	0	0	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0
1	1	1	0	1	0

2.[11 points] Programmable Machine

(a)[9 points] Below is the picture of a small processor and a FSM which computes

$$F = N + (N-1) + (N-2) + \dots + 2 + 1$$



Complete following tables

s	z	a_{SEL}	b_{SEL}	s'
00	0	11	00	01
00	1	11	00	01

s	z	a_{SEL}	b_{SEL}	s'
01	0	01	01	01
01	1	01	01	10

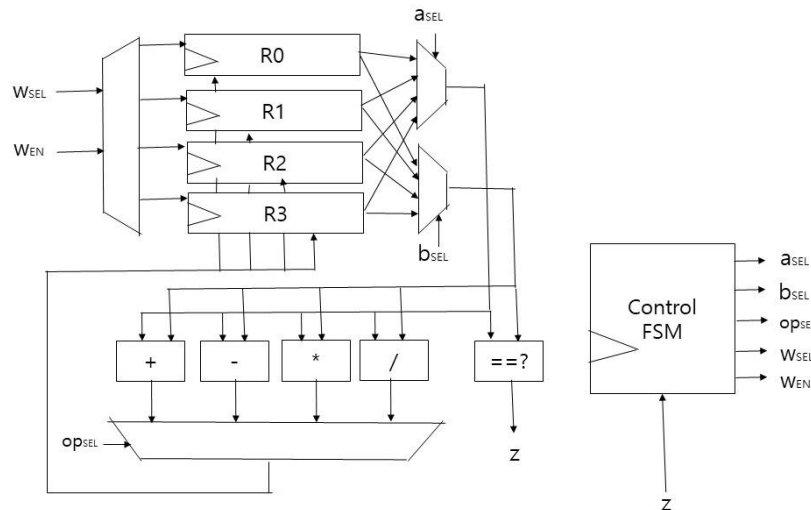
s	z	a_{SEL}	b_{SEL}	s'
10	0	00	10	10
10	1	00	10	10

(b)[20 points] Below is the picture of a small machine

STUDENT NAME:

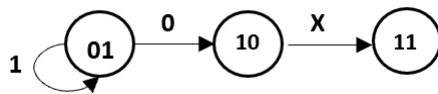
STUDENT ID NUMBER:

SECTION NUMBER IN THIS COURSE:



Initially, R0 contains N, R1 contains k, R2 contains -1, R3 contains 1. In the final state, R3 has final result. Finish the FSM to calculate $N ** K$ (exponential). Start state is '00' and final state is '11'.

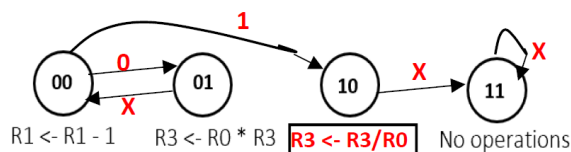
1. Write the operation of state '01' inside the rectangle.
2. Draw arrows, and write 'z' value (0 or 1) above each arrow. If 'z' value can be either 0 or 1, write 'X' above the arrow.



In the state '01' of the above example FSM, if $z == 0$, next state is '10', otherwise, next state is '01'.

In the state '10' of the above example FSM, next state is '11' regardless of the value of 'z'.

3. Finish following tables



s	z	op _{SEL}	w _{SEL}	w _{EN}	a _{SEL}	b _{SEL}	s'
00	0	00	01	1	01	10	01
00	1	00	01	1	01	10	10

s	z	op _{SEL}	w _{SEL}	w _{EN}	a _{SEL}	b _{SEL}	s'
01	0	10	11	1	00	11	00
01	1	10	11	1	00	11	00

s	z	op _{SEL}	w _{SEL}	w _{EN}	a _{SEL}	b _{SEL}	s'
10	0	11	11	1	11	00	11
10	1	11	11	1	11	00	11

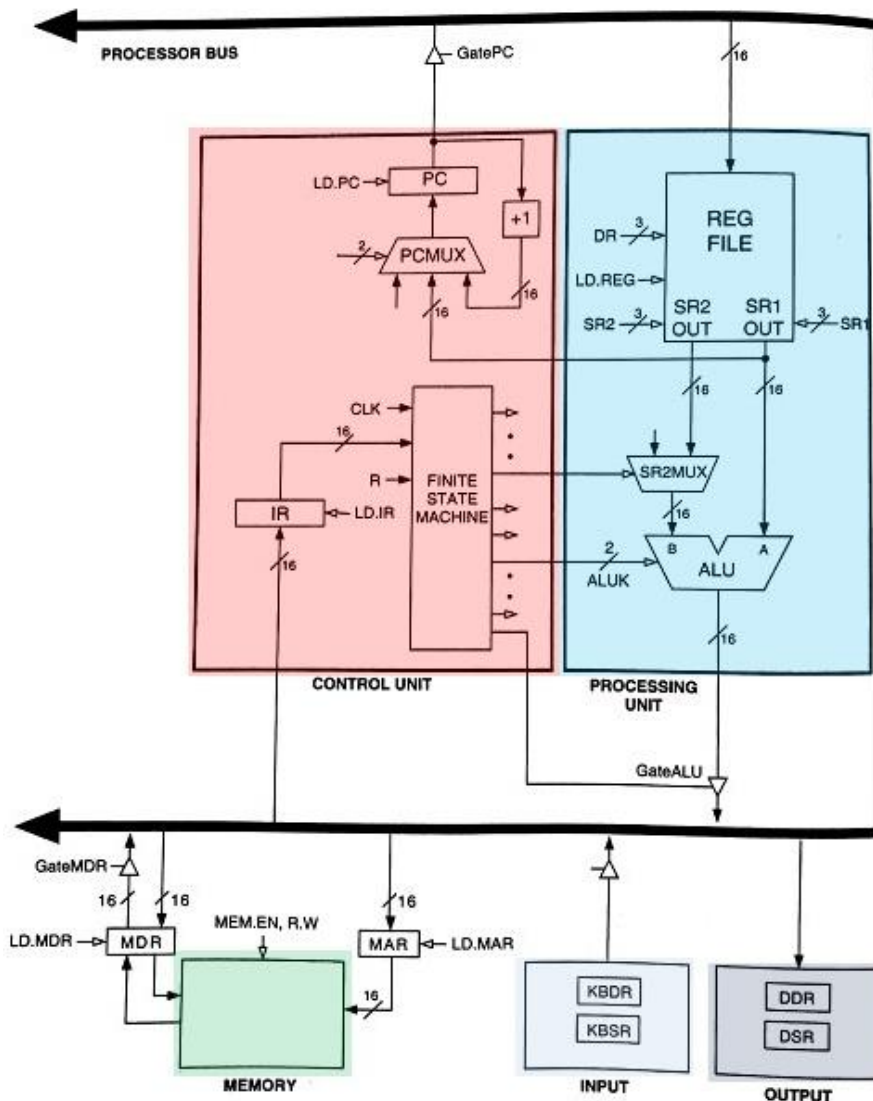
STUDENT NAME:

STUDENT ID NUMBER:

SECTION NUMBER IN THIS COURSE:

3.[9 points] LC-3 Architecture

Below is the picture of LC-3 processor



(a)[3 points] Fill in the values of control signals in the first instruction cycle? In that cycle, the following operations are processed at the same time.

MAR ← PC

PC ← PC + 1

GatePC	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	LD.MDR
1	0	0	0	1	1	0

(b)[3 points] Fill in the values of control signals in the STORE RESULT cycle? The instruction is R3 ← R2 + R1.

GateALU	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	DR
1	0	1	0	0	0	011

(c)[3 points] Fill in the values of control signals in the FETCH OPERAND cycle? The instruction is LDR R1 R2 #0

GatePC	GateMDR	LD.REG	LD.IR	LD.PC	LD.MAR	LD.MDR
0	0	0	0	0	0	1

STUDENT NAME:

STUDENT ID NUMBER:

SECTION NUMBER IN THIS COURSE:

4.[24 points] LC-3 Machine Language

LD (Load) $DR = M[PC + SEXT(PCOffset9)]$	ST (Store) $M[PC + SEXT(PCOffset9)] = SR$	LEA (Load Effective Address) $DR = PC + SEXT(PCOffset9)$	BR (Conditional Branch) If $((N \text{ AND } n) \text{ OR } (Z \text{ AND } z) \text{ OR } (P \text{ AND } p))$ $PC = PC + SEXT(PCOffset9)$
LI (Load Indirect) $DR = M[M[PC + SEXT(PCOffset9)]]$	STI (Store Indirect) $M[M[PC + SEXT(PCOffset9)]] = SR$	ADD (Addition) $DR = SR1 + SR2$	AND (Bit-wise Logical AND) $DR = SR1 \text{ AND } SR2$
LDR (Load Base + offset) $DR = M[BaseR + SEXT(offset6)]$	STR (Store Base + offset) $M[BaseR + SEXT(offset6)] = SR$	NOT (Load Base + offset) $DR = \text{NOT}(SR)$	 $DR = SR1 \text{ AND } SEXT(imm5)$

(a)[8 points] Finish LC-3 machine code to load R1 with integer 224 ($R1 \leftarrow 224$). Assume that your memory and registers are empty, and try to make it run as fast as possible. Write semantics of your instructions to the right of your instructions.

0	1	0	1	0	0	1	0	0	1	1	0	0	0	0	0	$R1 \leftarrow 0$
0	0	0	1	0	0	1	0	0	1	1	0	1	1	1	1	$R1 \leftarrow R1 + 15 \quad // R1 = 15$
0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	1	$R1 \leftarrow R1 + R1 \quad // R1 = 30$
0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	1	$R1 \leftarrow R1 + R1 \quad // R1 = 60$
0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	1	$R1 \leftarrow R1 + R1 \quad // R1 = 120$
0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	1	$R1 \leftarrow R1 + R1 \quad // R1 = 240$
0	0	0	1	0	0	1	0	0	1	1	1	0	0	0	0	$R1 \leftarrow R1 - 16 \quad // R1 = 224$

(b)[8 points] Now, R1 has integer 224 and integer k is stored in memory at location x30FF. Finish LC-3 machine code to calculate $224 * k$ and save the result to R0. Write semantics of your instructions to the right of your instructions when missing. Your instructions start at location x3000.

x3000	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	$R3 \leftarrow 0$
x3001	0	0	1	0	0	1	0	0	1	1	1	1	1	1	0	$R2 \leftarrow M[x30FF]$
x3002	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	If $(R2 == 0)$ Branch out of loop
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	1	$R3 \leftarrow R3 + R1$
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	$R2 \leftarrow R2 - 1$
x3005	0	0	0	0	1	1	1	1	1	1	1	1	1	1	0	Branch to start of loop
x3006	0	0	0	1	0	0	0	0	1	1	1	0	0	0	0	$R0 \leftarrow R3 + 0$

(c)[8 points] Above code may be inefficient when integer k is bigger than 224. Write machine code to make it more efficient by comparing 224 with k. Later you're going to insert this machine code between instructions at x3001 and x3002 without updating your code in problem (b)

x3002	1	0	0	1	1	0	1	0	1	0	1	1	1	1	1	$R4 \leftarrow R1 - R2$
x3003	0	0	0	1	1	0	1	1	0	1	1	0	0	0	1	(Use R5 to store the 2's complement of R2)
x3004	0	0	0	1	1	0	0	0	0	1	0	0	0	1	0	
x3005	0	0	0	0	0	1	1	0	0	0	0	0	0	0	1	If $(R4 \geq 0)$ Branch to x3009
x3006	0	0	0	1	1	0	1	0	0	1	1	0	0	0	0	Swap R1 and R2
x3007	0	0	0	1	0	0	1	0	1	0	1	0	0	0	0	(Use R5 for storing the temporary value)
x3008	0	0	0	1	0	1	0	1	0	1	1	0	0	0	0	
x3009	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	$R2 \leftarrow R2$

STUDENT NAME:

STUDENT ID NUMBER:

SECTION NUMBER IN THIS COURSE:

5.[16 points] LC-3 Machine Language

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	0	1	
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	0
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	1	0	1	1
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	0
x30FC	1	0	1	0	0	1	1	1	1	1	1	1	0	1	1	1

(a)[4 points] The PC has just become x30F8 at the Fetch phase. What is the value of R1?

R1 = x30F4

(b)[4 points] The PC has just become x30F9 at the Fetch phase. What is the value of R2?

R2 = x3102

(c)[4 points] The PC has just become x30FB at the Fetch phase. What is the value of R2?

R2 = 0

(d)[4 points] The PC has just become x30FC at the Fetch phase. What is the value of R2?

R2 = 5

6.[18 points] ISA and ISA Trade-offs

(a)[10 points] Circle Uarch or ISA corresponding to whether each of the following characteristics is better classified as “microarchitecture” or “ISA”:

Size of the general purpose registers: ISA

Load/store vs. memory/memory architectures: ISA

Support for virtual memory: ISA

The number of ALU’s in the CPU: Uarch

The number of general purpose registers in the CPU: ISA

Variable length instructions: ISA

Exception and interrupt handling: ISA

Access protection: ISA

Privilege modes: ISA

Special I/O instructions: ISA